

Strava Data Engineering & Visualization Project Plan

Updated with Gap Analysis

1. Project Overview

This project aims to analyze and visualize personal Strava activity data from GPX, FIT, and GZ files. The goal is to build an end-to-end data pipeline and an interactive dashboard to explore activities such as running, cycling, walking, and hiking.

Key outputs include:

- Interactive activity maps
- Heatmaps of all activities
- A Streamlit dashboard for exploration
- Advanced analytics and insights

2. Data Understanding

File Types

- **GPX**: XML-based, contains route and basic metrics
- **FIT**: Binary format, includes advanced metrics
- **GZ**: Compressed files (likely GPX or JSON)

Expected Data Fields

Time-Series Data

- Timestamp
- Latitude, Longitude
- Elevation
- Speed
- Distance
- Cadence

Physiological Data

- Heart rate
- Calories
- VO2 max (FIT files)

Activity Metadata

- Activity type (run, ride, hike)
- Activity title
- Moving time vs elapsed time

Environmental Data

- Temperature

Advanced Metrics (FIT)

- Power (cycling)
- Stride length
- Vertical oscillation

Initial Task

- Parse a small sample of files
- Identify all available fields
- Create a data dictionary

3. Data Pipeline Design

Ingestion Layer

Tools:

- gpxpy (GPX parsing)
- fitparse (FIT parsing)
- gzip (GZ handling)

Processing Layer

Standardize all data into a unified schema:

```
{ activity_id, activity_type, timestamp, lat, lon, elevation, heart_rate, speed, cadence }
```

Storage Options

- Parquet (recommended)
- SQLite / DuckDB
- Delta Lake (advanced option)

4. Visualization Features

A. Activity Maps

- Plot routes using folium or plotly
- Show start and end points
- Color routes by: Elevation, Heart rate, Speed

B. Heatmaps

- Global heatmap of all activities
- Activity-specific heatmaps (run, cycle, etc.)

C. Advanced Map Features

- Speed-based heatmap
- Elevation profile synchronization
- Route clustering

5. Streamlit Dashboard Design

Sidebar Filters

- Activity type
- Date range
- Distance
- Duration

Main Sections

Activity List: Title, Date, Distance, Duration

Activity Detail View — Metrics:

- Average heart rate
- Maximum heart rate
- Elevation gain
- Pace

Heatmap Section

- Overall heatmap
- Activity-type heatmap

Trends Section

- Weekly/monthly distance

- Pace trends
- Heart rate trends

6. Advanced Features

- **Personal Records Tracker:** Fastest 5K, 10K; Longest activity; Best pace
- **Effort Score:** $\text{effort} = \text{avg_hr} \times \text{duration} \times \text{elevation_gain}$
- **Route Reuse Detection:** Identify frequently used routes
- **Activity Classification:** Predict activity type using speed and cadence
- **Calendar Heatmap:** Activity frequency visualization
- **Streak Tracker:** Longest activity streaks
- **Segment Analysis:** Uphill vs downhill performance, sprint detection

7. Tech Stack

- Parsing: gpxpy, fitparse
- Processing: pandas, numpy
- Storage: parquet, duckdb
- Visualization: folium, plotly, altair
- Dashboard: Streamlit

8. Project Structure

```
strava-project/
├── data/
│   ├── raw/
│   ├── processed/
│   └── ingestion/
├── parse_gpx.py
├── parse_fit.py
├── processing/
├── transform.py
├── features/
├── effort_score.py
├── pr_detection.py
├── app/
├── streamlit_app.py
├── notebooks/
└── exploration.ipynb
```

9. Smart Enhancements

- Map matching (snap routes to roads)
- Weather data integration
- Indoor vs outdoor comparison
- Fatigue score over time

10. Pitfalls to Avoid

- Skipping schema standardization
- Handling FIT parsing incorrectly

- Not optimizing for large datasets
- Rendering too many points on maps

11. Suggested Build Order

1. Parse GPX and visualize one route
2. Create unified schema
3. Store processed data in Parquet
4. Build heatmap
5. Develop Streamlit dashboard
6. Add advanced features

12. Future Scope

- Real-time activity ingestion
- Integration with Strava API
- Social sharing features
- Mobile-friendly dashboard

13. Gap Analysis & Recommendations

The following gaps were identified in the original project plan. Addressing these will make the pipeline more robust, the project more maintainable, and the dashboard more useful.

13.1 Data & Schema Gaps

Schema drops FIT-specific fields

The unified schema defined in Section 3 omits several fields captured in Section 2: power, stride_length, vertical_oscillation, temperature, calories, and vo2_max. These would be silently lost during transformation.

Recommendation: Extend the schema to include all known fields, using nullable columns for data that may not be present in all file types:

```
{ activity_id, activity_type, timestamp, lat, lon, elevation, heart_rate, speed,
  cadence, power, stride_length, vertical_oscillation, temperature, calories,
  vo2_max }
```

No handling for missing or null fields

Many activities will be recorded without a heart rate monitor, power meter, or GPS. The pipeline must not fail or silently drop rows when optional fields are absent.

- Define which fields are required vs. optional in the schema
- Use nullable types in Parquet/DuckDB for all optional fields
- Add a post-parse check that logs which fields are missing per file

No data validation or quality checks

Corrupt, partial, or malformed files could produce bad data that flows silently into the dashboard. Recommend adding a validation layer:

- Check for required fields after parsing (activity_id, timestamp, lat/lon)
- Validate coordinate ranges (lat: -90 to 90, lon: -180 to 180)
- Flag activities with suspiciously short duration or zero distance
- Use pandera or Great Expectations for schema enforcement

13.2 Pipeline & Infrastructure Gaps

GZ file contents not specified

GZ files are listed as a supported format, but the plan never specifies what is inside them (GPX, JSON, or FIT). The ingestion layer needs to detect content type after decompression:

```
import gzip, magic with gzip.open(file, 'rb') as f: content = f.read() file_type
= magic.from_buffer(content) # detect GPX vs JSON vs FIT
```

No error handling or retry strategy

If a file fails to parse, the pipeline currently has no defined behaviour. Recommend:

- Wrap each file parse in a try/except block
- Log failures to a separate error log with filename, error type, and traceback
- Move failed files to a data/failed/ quarantine directory
- Continue processing remaining files rather than halting the entire run

No batching or chunking strategy for large datasets

Section 10 warns against not optimizing for large datasets, but provides no solution. For users with thousands of activities:

- Process files in configurable batch sizes (e.g. 100 at a time)
- Use incremental loading — track which files have already been parsed
- Store a processed_files manifest to avoid re-parsing on each run
- Consider Dask or Polars for out-of-memory DataFrame operations

No testing plan

There are no unit or integration tests described. Recommend adding a tests/ directory:

- Unit tests for each parser (parse_gpx.py, parse_fit.py) with sample fixture files
- Schema validation tests to confirm output matches the unified schema
- Integration test that runs the full pipeline on a small sample dataset
- Use pytest as the test runner

13.3 Dashboard Gaps

Missing filter dimensions

The sidebar filters (Section 5) omit several dimensions that athletes commonly use. Recommend adding:

- Heart rate zone filter (Z1–Z5)
- Elevation gain range filter
- Gear / shoe tracking filter (if available in metadata)
- Personal record flag (filter to PR activities only)

No per-activity charts in Detail View

The Activity Detail View (Section 5) lists summary metrics but no time-series charts. Recommend adding:

- Pace or speed over distance/time chart
- Heart rate over distance/time chart
- Elevation profile chart (with grade coloring)
- Synchronized map + chart interaction (hover on chart highlights map position)

No performance or loading strategy for large data

As the dataset grows, the Streamlit app will slow down. Recommend:

- Use st.cache_data to cache loaded DataFrames
- Load only summary data on initial page render; fetch detail on demand
- Limit map rendering to a maximum of ~5,000 GPS points per route (downsample with rdp algorithm)
- Add a loading spinner for long-running computations

13.4 Project Management Gaps

No timeline or effort estimates

The build order in Section 11 provides a sequence but no time estimates. Recommend adding a rough milestone plan:

- Week 1: Parse GPX + visualize one route
- Week 1–2: Unified schema + Parquet storage
- Week 2–3: Full ingestion pipeline (GPX, FIT, GZ)

- Week 3: Heatmap
- Week 4–5: Streamlit dashboard (core)
- Week 6+: Advanced features

Strava data export not documented

The plan assumes activity files already exist locally, but first-time setup is non-trivial. Recommend adding a setup section:

- Go to Strava Settings → My Account → Download or Delete Your Data
- Request and download the export archive (may take up to 24 hours)
- Unzip and place activity files in data/raw/
- Run the ingestion pipeline to process all files

Strava API integration lacks detail

Section 12 lists Strava API integration as future scope, but it is significantly more complex than a bullet point implies:

- Requires OAuth 2.0 authentication flow with access + refresh tokens
- Rate limited to 100 requests/15 min and 1,000 requests/day
- Activity streams endpoint needed for GPS/HR data (not just summary metadata)
- Consider using stravalib as a Python wrapper

13.5 Project Structure Gaps

Missing standard project files

The folder structure in Section 8 is missing several files expected in a real portfolio project:

```
strava-project/ ■■■ tests/ ← NEW: unit + integration tests ■ ■■■
test_parse_gpx.py ■ ■■■ test_parse_fit.py ■ ■■■ test_schema.py ■■■ config/ ←
NEW: configuration ■ ■■■ settings.yaml ■■■ logs/ ← NEW: pipeline run logs ■■■
data/ ■ ■■■ raw/ ■ ■■■ processed/ ■ ■■■ failed/ ← NEW: quarantine for parse
failures ■■■ requirements.txt ← NEW: dependency pinning ■■■ README.md ← NEW:
setup + usage instructions
```

No logging or observability plan

When the pipeline breaks on a specific file, there is currently no way to diagnose it. Recommend:

- Add Python logging (logging module) to all pipeline scripts
 - Log at INFO level: files processed, rows written, time taken
 - Log at WARNING level: missing optional fields, low data quality
 - Log at ERROR level: parse failures with full traceback
 - Write logs to logs/pipeline_YYYYMMDD.log with rotation
-

Conclusion

This project combines data engineering, analytics, and visualization to create a comprehensive fitness tracking system. By following a structured approach and addressing the gaps identified in Section 13, it can serve as a strong portfolio project demonstrating real-world data engineering skills — including robust pipeline design, data quality, testing, and production-ready application development.